

Terminal-output For UCI SMS Spam Multinomial Naive Bayes Classifier

```
alex.omegapy@MacStudio CSC510-CTAs % .venv/bin/python CTA_Module-6/sms_spam_naive_bayes.py --demo
--no-pause

=====
UCI SMS Spam Multinomial Naive Bayes Classifier
=====

Course           : CSC510 Module 6 Critical Thinking Assignment
Program version  : 1.0.0
Python           : 3.12.13
NumPy            : 2.5.1
scikit-learn     : 1.9.0

=====
1. TASK, MODEL, PARAMETERS, AND EQUATIONS
=====

This program sorts an SMS message into one of two groups: HAM or SPAM. CountVectorizer
first learns the words used in the UCI SMS dataset. It then turns each message into word
counts. The dataset contains 5,574 messages. Of these, 4,825 are HAM, which means non-
spam, and 747 are SPAM.
The program trains MultinomialNB(alpha=1.0) with those word counts. During training, the
model learns how common HAM and SPAM messages are and how often each word appears in
each group. Laplace smoothing keeps a word with a zero count from making the probability
for that whole group zero.
When the user enters a message, or the demo supplies one, CountVectorizer counts only
words learned during training. The program then calculates the chance that the message
is HAM and the chance that it is SPAM. It performs the calculation step by step and
compares it with scikit-learn's predict_proba() result. The class with the higher
probability becomes the prediction.
The terminal shows each part of the calculation so the final result is easier to follow.

Problem definition
-----
Learning setting      : Supervised learning
Decision              : Binary text classification
Candidate classes     : HAM and SPAM
Input x               : One SMS message represented as word counts
Feature values xj   : Discrete nonnegative occurrence counts
Why not regression    : The required output is a class label

Why Multinomial Naive Bayes works well with word counts
-----
CountVectorizer counts how many times each word appears in a message. Multinomial Naive
Bayes works well with these word counts, which is why it is often used to classify text.
Gaussian Naive Bayes is a better fit for continuous measurements. Bernoulli Naive Bayes
only checks whether a word appears, so it ignores repeated words.
Naive Bayes makes one simplifying assumption. When it scores a message as spam or not
spam, it treats each word as if it were unrelated to the other words. Real language does
not work exactly this way, but this shortcut lets the model combine what it learned
about each word into one score for each class.

scikit-learn configuration
-----


| Component       | Parameter          | What it controls                                    |
|-----------------|--------------------|-----------------------------------------------------|
| CountVectorizer | lowercase=True     | Convert tokens to lowercase                         |
| CountVectorizer | binary=False       | Retain repeated word counts                         |
| CountVectorizer | ngram_range=(1, 1) | Use one-word features                               |
| MultinomialNB   | alpha=1.0          | Apply Laplace add-one smoothing                     |
| MultinomialNB   | fit_prior=True     | Estimate P(C <sub>k</sub> ) from M <sub>k</sub> / M |
| MultinomialNB   | force_alpha=True   | Use alpha exactly as supplied                       |

Bayes' theorem, class ranking, and MAP
```

$$P(C_k | x) = P(x | C_k) * P(C_k) / P(x)$$

For the same input x , $P(x)$ is common to HAM and SPAM:
 $P(C_k | x)$ is proportional to $P(x | C_k) * P(C_k)$

$$C_{\text{hat}} = \text{argmax}_k P(C_k | x)$$

C_k = candidate class k , HAM or SPAM
 x = input word-count vector
 $P(C_k)$ = prior before the input words are considered
 $P(x | C_k)$ = likelihood of those word counts under class C_k
 $P(C_k | x)$ = posterior after prior and word evidence are combined
 $P(x)$ has the same value for HAM and SPAM, so it does not change which class gets the higher score. The program still turns the scores into probabilities so the two results add up to one.

Frequency, likelihood, score, and posterior equations

Class prior from message frequencies:
 $P(C_k) = M_k / M$

Raw word-frequency estimate:
 $P_{\text{MLE}}(w_j | C_k) = N_{jk} / N_k$

Laplace-smoothed estimate used by MultinomialNB:
 $P(w_j | C_k) = (N_{jk} + \alpha) / (N_k + \alpha * V)$

Message likelihood and unnormalized class score:
 $P(x | C_k) = \text{product}_j P(w_j | C_k) ** x_j$
 $\text{score}_k = P(C_k) * P(x | C_k)$

Equivalent log-space calculation used by the program:
 $\log P(x | C_k) = \text{sum}_j x_j * \log P(w_j | C_k)$
 $\log \text{score}_k = \log P(C_k) + \log P(x | C_k)$

Stable normalization:
 $r_k = \exp(\log \text{score}_k - \max_l \log \text{score}_l)$
 $P(C_k | x) = r_k / \text{sum}_l r_l$

Equation symbols and source-code variables

Symbol	Source variable	Meaning
M	<code>m_total_messages</code>	All labeled training messages
M_k	<code>m_k_class_message_count</code>	Training messages in class k
V	<code>v_vocabulary_size</code>	Number of learned word features
N_{jk}	<code>n_jk_word_count_by_class</code>	Training count of word j in class k
N_k	<code>n_k_total_word_count_by_class</code>	All modeled word counts in class k
x_j	<code>x_j_input_word_counts</code>	Input-message count for word j
θ_{jk}	<code>theta_hat_jk_word_likelihood</code>	Smoothed $P(\text{word } j \text{class } k)$
$P(C_k x)$	<code>p_c_k_given_x_posterior</code>	Normalized class posterior
argmax	<code>k_predicted_class_index</code>	Index of the largest posterior

Inside `calculate_posterior_probabilities()`, `x_input_sparse.dot(feature_log_prob.T)` adds up how much the message's words support HAM and SPAM. Then `np.argmax` picks the class with the higher final probability.

2. DATASET AND FREQUENCY TABLES

Dataset mode : COMPLETE UCI DATASET
Dataset path : data/SMS SpamCollection
Decoded as : utf-8-sig
 M , total messages : 5,574
 V , vocabulary size : 8,713

M_k is the number of messages in each class, and the program uses it to find each class prior. N_k and N_{jk} are word counts used to find the likelihood for each word. Message counts and word counts have different jobs in the calculation.

Class counts, priors, and class word totals

$P(C_k) = M_k / M$ $N_k = \sum_j N_{jk}$			
Class	M_k messages	Prior P(C_k)	N_k word tokens
HAM	4,827	0.86598493	62,965
SPAM	747	0.13401507	17,487
TOTAL	5,574	1.00000000	80,452

M_k counts the labeled messages in one class. The formula $P(C_k)=M_k/M$ uses that count to find the class prior. N_k counts all word occurrences in that class. It belongs in the denominator of $P(w_j | C_k)$ and is not another message count.

Examples from the class-by-word frequency table

Class	Training word w_j	Count N_jk
HAM	1. you	1,948
HAM	2. to	1,562
HAM	3. the	1,133
HAM	4. and	858
HAM	5. in	823
HAM	6. me	777
SPAM	1. to	691
SPAM	2. call	355
SPAM	3. you	297
SPAM	4. your	264
SPAM	5. free	224
SPAM	6. the	206

The table shows only a few of the most common words. The model still keeps the full table of V words and uses every learned word that appears in the new message.

3. INPUT WORD COUNTS, LIKELIHOODS, AND LAPLACE CORRECTION

Input message

"Dear Valued Customer, Congratulations! Your mobile number has been selected to receive a cash prize worth one thousand dollars. Please call our claims department today and provide your confirmation code to collect your reward before this limited-time offer expires. Sincerely, Rewards Center."

Here, CountVectorizer works on the new message only. Its x_j values show how many times each known word appears in that message. The N_{jk} values come from the training messages and show how often those words appeared in each class.

Input vector x: nonzero word counts

Vocabulary word w_j	Input count x_j
and	1
been	1
before	1
call	1
cash	1
center	1
claims	1
code	1
collect	1
congratulations	1
customer	1
dear	1
department	1
dollars	1
expires	1
has	1
to	2
your	3

The table only needs to show words with $x_j > 0$. When $x_j = 0$, the word adds $0 * \log$

$P(w_j | C_k) = 0$ to both class scores.
 16 more nonzero input terms are left out of the table to keep it within 92 columns. The calculation still uses those terms.

Class-conditional likelihoods used for this message

Raw estimate: $P_{MLE}(w_j | C_k) = N_{jk} / N_k$
 Fitted estimate: $P(w_j | C_k) = (N_{jk} + \alpha) / (N_k + \alpha * V)$
 $\alpha = 1.0$

Word	x_j	$N_{j,HAM}$	$P(w HAM)$	$N_{j,SPAM}$	$P(w SPAM)$
and	1	858	0.01198415	122	0.00469466
been	1	88	0.00124166	43	0.00167939
before	1	61	0.00086498	5	0.00022901
call	1	238	0.00333436	355	0.01358779
cash	1	12	0.00018137	76	0.00293893
center	1	2	0.00004185	2	0.00011450
claims	1	0	0.00001395	2	0.00011450
code	1	1	0.00002790	30	0.00118321
collect	1	5	0.00008371	23	0.00091603
congratulations	1	1	0.00002790	14	0.00057252
customer	1	9	0.00013951	49	0.00190840
dear	1	106	0.00149279	16	0.00064885
department	1	2	0.00004185	0	0.00003817
dollars	1	5	0.00008371	0	0.00003817
expires	1	0	0.00001395	17	0.00068702
has	1	87	0.00122771	31	0.00122137
to	2	1562	0.02180585	691	0.02641221
your	3	417	0.00583164	264	0.01011450

Each $P(w_j | C_k)$ describes how likely one word is for one class. It is only one part of the calculation, not the final $P(C_k | x)$. The model combines all of these word likelihoods with x_j and the class prior.

A real zero count and its Laplace correction

Example word w_j : claims
 Example class C_k : HAM
 Observed training count N_{jk} : 0
 Raw relative-frequency estimate:
 $P_{MLE}(\text{claims} | \text{HAM})$
 $= 0 / 62965$
 $= 0.000000000000$

Laplace estimate used by the fitted model:
 $P(\text{claims} | \text{HAM})$
 $= (0 + 1)$
 $/ (62965 + 1 * 8713)$
 $= 0.000013951282$

The word still has a count of zero for this class. Laplace smoothing does not change that count. It only gives the word a small positive likelihood so one unseen word cannot reduce the entire class score to zero.

4. MANUAL NAIVE BAYES POSTERIOR CALCULATION

From message likelihood to posterior

$\log P(x | C_k) = \sum_j x_j * \log P(w_j | C_k)$
 $\log_score_k = \log P(C_k) + \log P(x | C_k)$

$r_k = \exp(\log_score_k - \max_l \log_score_l)$
 $P(C_k | x) = r_k / \sum_l r_l$

Multiplying many very small word probabilities can produce a number too small for the computer to store accurately. The program uses logarithms to turn the multiplication into addition. It also shifts both scores by the same amount before exponentiation. These steps keep the calculation stable without changing which class wins.

Class HAM

```
-----
log_score_HAM =
  log P(HAM)
  + 1 * log P(and | HAM)
  + 1 * log P(been | HAM)
  + 1 * log P(before | HAM)
  + 1 * log P(call | HAM)
  + 1 * log P(cash | HAM)
  + 1 * log P(center | HAM)
  + 1 * log P(claims | HAM)
  + 1 * log P(code | HAM)
  + 1 * log P(collect | HAM)
  + 1 * log P(congratulations | HAM)
  + 1 * log P(customer | HAM)
  + 1 * log P(dear | HAM)
  + 1 * log P(department | HAM)
  + 1 * log P(dollars | HAM)
  + 1 * log P(expires | HAM)
  + 1 * log P(has | HAM)
  + 2 * log P(to | HAM)
  + 3 * log P(your | HAM)
  + [remaining modeled input terms]
Log prior, log P(HAM)           : -0.143887772381
Message log likelihood          : -293.441058400393
Unnormalized class log score    : -293.584946172775
Shifted relative score r_k      : 0.000000000000
Posterior P(HAM | x)           : 0.000000000000
```

Class SPAM

```
-----
log_score_SPAM =
  log P(SPAM)
  + 1 * log P(and | SPAM)
  + 1 * log P(been | SPAM)
  + 1 * log P(before | SPAM)
  + 1 * log P(call | SPAM)
  + 1 * log P(cash | SPAM)
  + 1 * log P(center | SPAM)
  + 1 * log P(claims | SPAM)
  + 1 * log P(code | SPAM)
  + 1 * log P(collect | SPAM)
  + 1 * log P(congratulations | SPAM)
  + 1 * log P(customer | SPAM)
  + 1 * log P(dear | SPAM)
  + 1 * log P(department | SPAM)
  + 1 * log P(dollars | SPAM)
  + 1 * log P(expires | SPAM)
  + 1 * log P(has | SPAM)
  + 2 * log P(to | SPAM)
  + 3 * log P(your | SPAM)
  + [remaining modeled input terms]
Log prior, log P(SPAM)         : -2.009803022909
Message log likelihood          : -249.557800282603
Unnormalized class log score    : -251.567603305512
Shifted relative score r_k      : 1.000000000000
Posterior P(SPAM | x)          : 1.000000000000
```

Normalization check

```
-----
Largest class log score         : -251.567603305512
Sum of shifted scores           : 1.000000000000
Sum of manual posteriors        : 1.000000000000
r_k is a temporary score, not a probability yet. The program divides each r_k by the
total of both scores. This produces the two posterior probabilities and makes them add
up to one.
```

5. SCIKIT-LEARN COMPARISON AND FINAL PREDICTION

```

=====
Both columns come from the same trained MultinomialNB model. The manual column shows the
calculation steps, while predict_proba() shows scikit-learn's answer. If the values
match, the program's probability calculation is working as intended. This comparison
does not measure how well the model classifies new labeled messages.
Class | Manual P(C_k|x) | sklearn P(C_k|x) | Absolute difference
-----|-----|-----|-----
HAM | 0.000000000000 | 0.000000000000 | 0.000e+00
SPAM | 1.000000000000 | 1.000000000000 | 0.000e+00

Largest probability difference : 0.000e+00
Allowed numerical tolerance : 1.0e-12
Manual calculation matches : PASS
PASS means the manual probability for each class matches predict_proba() within the
small rounding difference allowed above.

Maximum a posteriori (MAP) decision
-----
C_hat = argmax_k P(C_k | x)
argmax returns k, the class index with the largest posterior.
P(HAM | message) : 0.000000000000
P(SPAM | message) : 1.000000000000
Predicted class : SPAM
Winning posterior : 1.000000000000
Winning percentage : 100.00%
The model compares the two posterior probabilities and picks the larger one. argmax does
this comparison. It does not use an extra cutoff for SPAM.

Final classification summary
-----
Input message: "Dear Valued Customer, Congratulations! Your mobile number has been
selected to receive a cash prize worth one thousand dollars. Please call our claims
department today and provide your confirmation code to collect your reward before this
limited-time offer expires. Sincerely, Rewards Center."
HAM percentage : 0.00%
SPAM percentage : 100.00%
Conclusion: It is SPAM.
alex.omegapy@MacStudio CSC510-CTAs %

```